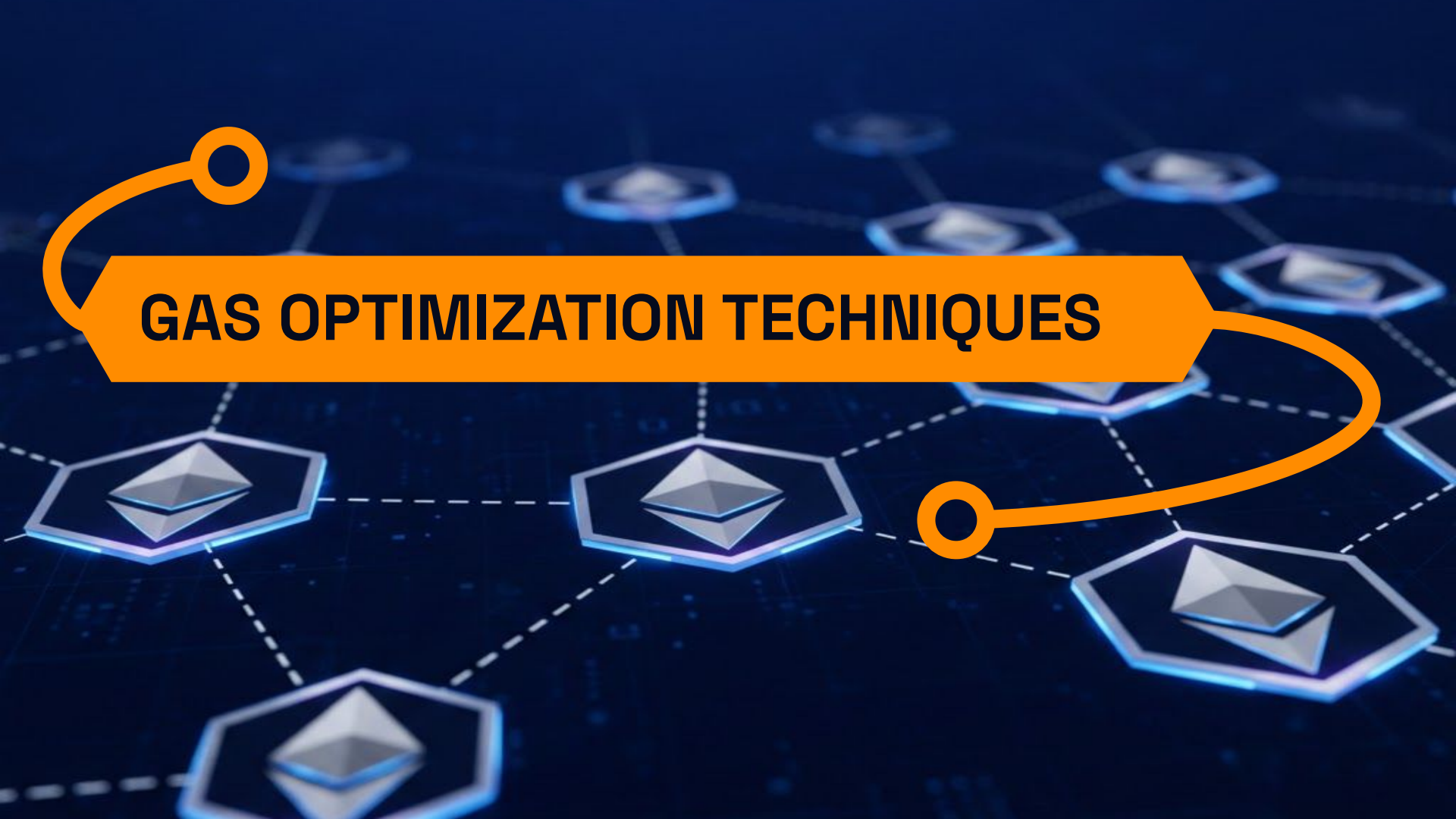
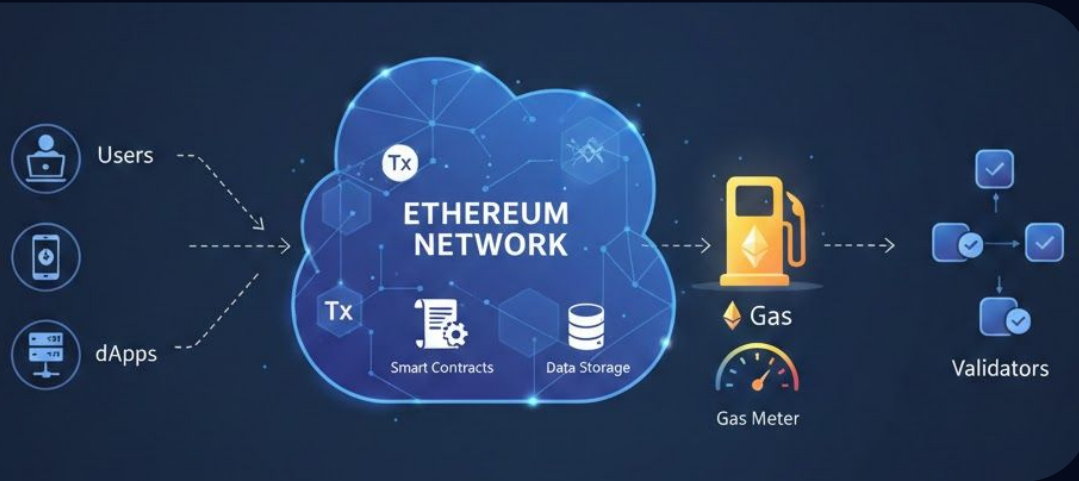


The background is a dark blue field with a network of glowing blue hexagons connected by dashed white lines. Each hexagon contains a 3D Ethereum logo. An orange callout box with a curved line and circular endcaps is positioned in the upper left, framing the title text.

GAS OPTIMIZATION TECHNIQUES

1. Gas Fundamentals & the EVM Cost Model

What is Gas in Ethereum ?

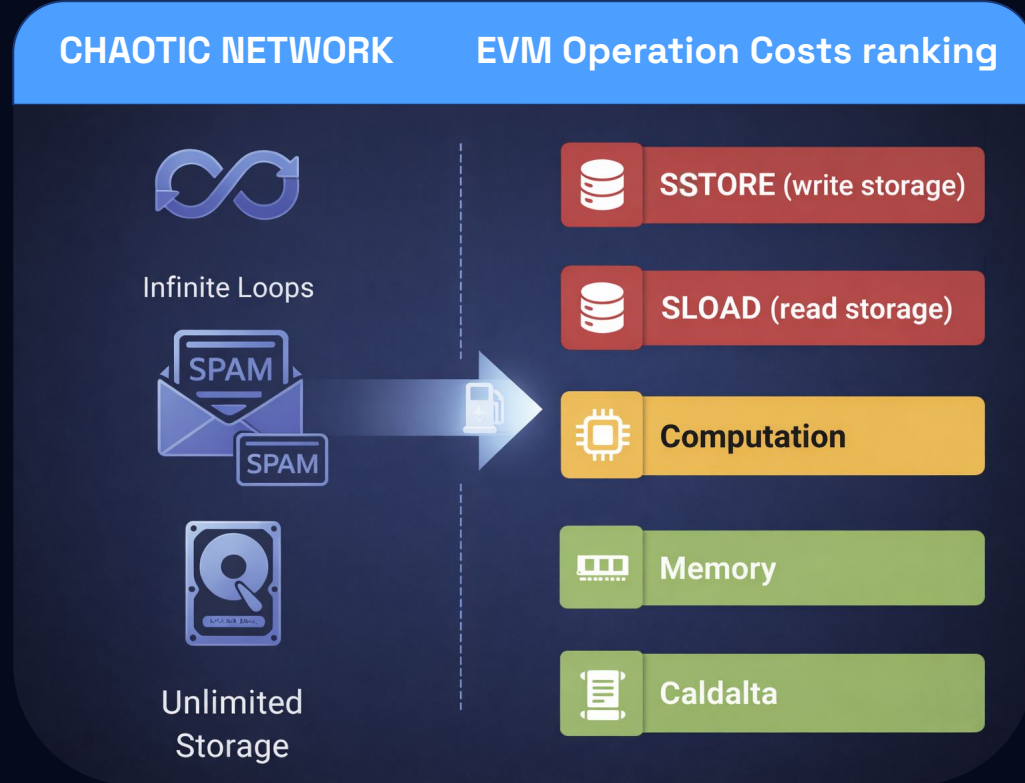


Gas = Unit of
Computational Effort

- Measures EVM work
- Paid in ETH
- Prevents spam

$\text{Gas Used} \times \text{Gas Price} = \text{ETH}$

1. Gas Fundamentals & the EVM Cost Model



Why Gas exists ?

Types of gas costs :

- Deployment gas
- Execution gas

1. Gas Fundamentals & the EVM Cost Model



Storage vs Memory vs Calldata

- Storage → permanent, most expensive
- Memory → temporary during function execution
- Calldata → read-only external input, cheaper than memory

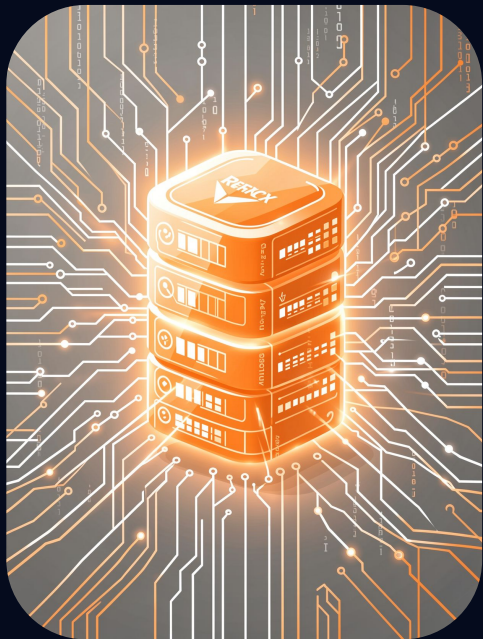
Hot Paths vs Rarely Used Functions

Block gas limit & Scalability

Gas refunds possibility

2. Solidity-Level Optimisations

Storage & Data Layout Optimisation



Storage Read Caching:

- Copy storage variables to local memory once.
- Reuse to avoid repeated SLOAD.
- Important for loops and hot paths.

2. Solidity-Level Optimisations

Storage & Data Layout Optimisation

Demo

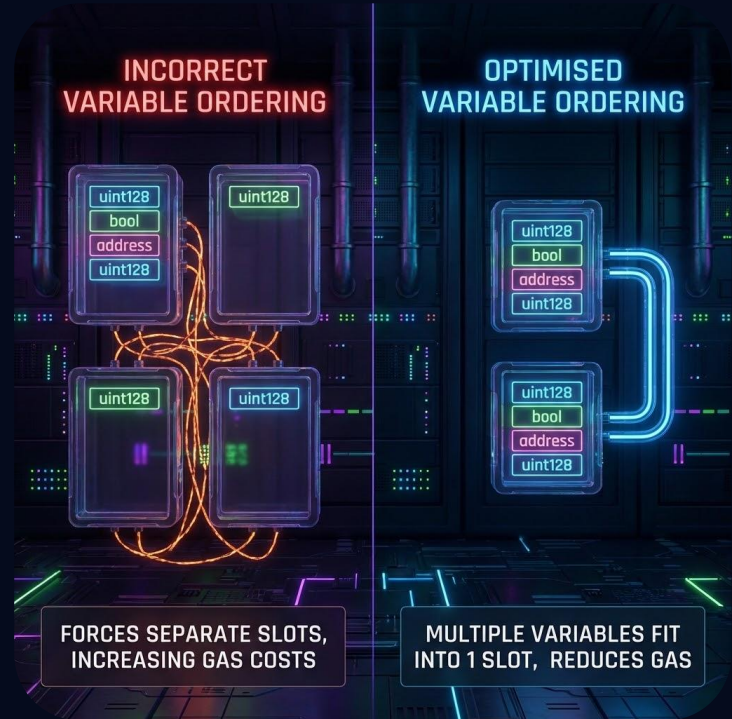


2. Solidity-Level Optimisations

Storage & Data Layout Optimisation

Struct Packing & Variable Ordering:

- Solidity uses 32-byte storage slots.
- Order smaller types (e.g., uint128, bool) correctly to fit multiple variables into 1 slot.



2. Solidity-Level Optimisations

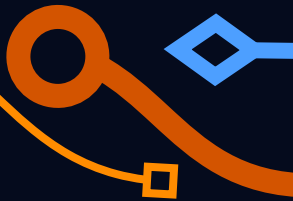
Storage & Data Layout Optimisation

Choosing Data Types:

Smaller integer types don't reduce computation gas.

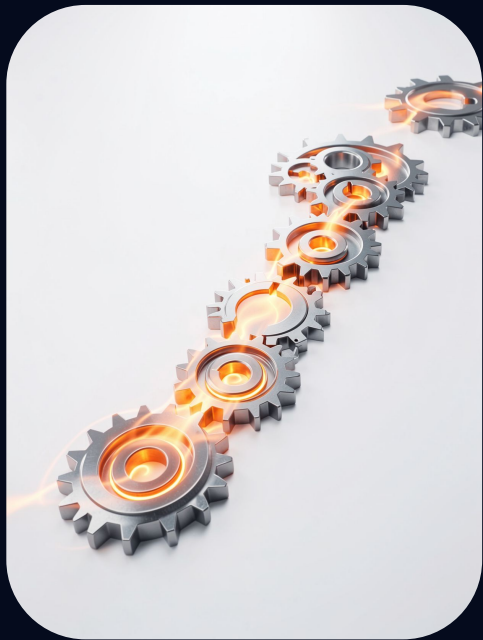
Type selection primarily impacts storage layout.

Optimisation requires understanding EVM mechanics.



2. Solidity-Level Optimisations

Control Flow & Arithmetic



Loop Optimisations:

Loops can be gas-intensive.

2 key improvements:

- Cache array length: `uint256 length = arr.length`
- Avoid unnecessary storage reads within loops.

These changes reduce repeated operations per iteration.

2. Solidity-Level Optimisations

Control Flow & Arithmetic

Unchecked Blocks:

Solidity 0.8+ adds automatic overflow checks.

- Use unchecked { i++; } when overflow is impossible (e.g., loop counters).
- Skip checks → slightly lower gas.



2. Solidity-Level Optimisations

Control Flow & Arithmetic

Conditional Ordering & Short-Circuiting:

Logical operators (&&) short-circuit.

- Place cheaper checks first.
- Place more likely-to-fail checks first.

Example:

```
if (x != 0 && expensiveFunction()) {}  
skips expensive call when x == 0.
```



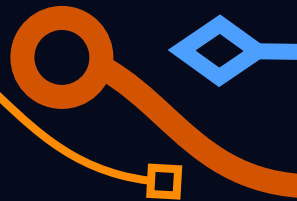
2. Solidity-Level Optimisations

Memory vs. Calldata demo



3. Advanced Patterns, Real-World Optimizations & Assembly

- Cover advanced Solidity patterns
- Show how production libraries optimize gas
- Introduce Assembly as a bonus tool.



3. Storage Variable vs Constant & Immutable

Concept

- Both avoid expensive storage reads.
- constant: value embedded into bytecode
- immutable: stored once at deployment, then like constant

Gas Saving

Reading storage (SLOAD) costs ~2100 gas;
constants cost almost nothing.

Example

BAD Storage variable: `uint256 public fee = 100;` (Every read costs gas)

GOOD Constant: `uint256 public constant FEE = 100;` (No storage read at runtime)

Use Case	Choose
<u>Known only in constructor</u>	immutable
<u>Known at compile time</u>	constant

3. Function Visibility: external vs public

Concept

external functions read parameters directly from calldata.

public functions copy them into memory.

Gas Saving

Avoiding memory allocation and copying reduces execution cost.

Rule of Thumb

If only called from outside the contract, use external.

Example

✗ More expensive:

```
function set(uint256[] memory data) public { }
```

✓ Cheaper:

```
function set(uint256[] calldata data) external { }
```

3. Reducing Bytecode Size

Deployment gas depends heavily on contract bytecode size.

01

Remove Unused Code

Erase dead code to shrink contract size.

02

Prefer Libraries

Use shared libraries over duplicating logic - Don't reinvent the wheel (at a higher gas cost)

03

Custom Errors

Use custom errors instead of long revert strings.

Example: Revert String vs Custom Error

✗ Expensive: `require(x > 0, "Value must be positive");`

✓ Cheaper: `error ValueMustBePositive(); if (x <= 0) revert ValueMustBePositive();`

Custom errors encode only a selector, saving thousands of deployment gas.

3. Real-World Optimisation :

OpenZeppelin's Non-Zero Boolean Pattern

The Observation

OpenZeppelin contracts avoid simple booleans (e.g., `bool locked;` in favour of `uint256 private constant _NOT_ENTERED = 1;` and `_ENTERED = 2;`

The EVM Storage Rule That Matters

<u>Transition</u>	<u>Cost</u>
Zero → Non-zero	Very expensive
Non-zero → Zero	Expensive
Non-zero → Non-zero	Cheaper

What happens with a normal boolean?

`locked = true;` ($0 \rightarrow 1$)

`locked = false;` ($1 \rightarrow 0$)

Repeatedly toggles between zero and non-zero, incurring most expensive storage costs every time.

3. OpenZeppelin's Solution

Never use zero after initialisation.

```
_status = 1; // NOT_ENTERED
```

```
_status = 2; // ENTERED
```

```
_status = 1; // NOT_ENTERED
```

The Impact

Demo

All writes become non-zero → non-zero, which is cheaper.

Why this matters

This modifier may run on every transaction.

Saving ~100-200 gas per call becomes huge with high volume.

OpenZeppelin intentionally avoids zero values in frequently updated storage variables, because non-zero → non-zero writes are cheaper than zero transitions.

3. Assembly (Yul)

What Assembly Is

- Write code closer to EVM opcodes.
- Skip safety checks.
- Optimise loops and storage manually.

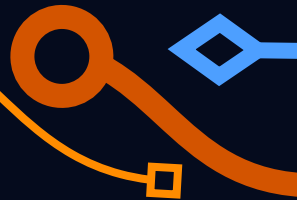
Why it can reduce gas

Solidity adds:

- Overflow checks
- Bounds checks
- Generic memory handling

Assembly lets you:

- Use direct opcodes
- Skip redundant work
- Produce smaller bytecode



3. Assembly: Simple Comparison Example



Solidity

```
function add(uint256 a, uint256 b) external  
pure returns (uint256) { return a + b; }
```

Assembly

```
function add(uint256 a, uint256 b) external  
pure returns (uint256 result) { assembly {  
    result := add(a, b) } }
```

3. What you'll do with Yul (Assembly)



Proxy Contracts

For upgradability.

Highly Optimised Loops

When Solidity's loop handling is too inefficient.

Cryptography Libraries

Implementing custom hash functions or signatures.

Calldata Encoding/Decoding

Efficient manipulation of raw transaction data.

3. Risks of Assembly



Assembly removes crucial safeguards present in Solidity:

**No Overflow
Protection**

**Lack of Type
Safety**

**Memory
Vulnerabilities**

Reduced Readability

One bug can permanently break a contract.



Thank You !

Bibliography



1. Tebrean, G. (2024, July 10). *Solidity Smart Contracts: Gas Optimization Techniques*. Web3 Labs Blog.
<https://blog.web3labs.com/solidity-smart-contracts-gas-optimization-techniques/>
2. Ancilar | Blockchain Services. (2025, October 10). *5 Gas Optimization Patterns Every Solidity Developer Must Master*. Medium; Coinmonks.
<https://medium.com/coinmonks/5-gas-optimization-patterns-every-solidity-developer-must-master-f25890d239e6>
3. *Gas Optimization In Solidity: Strategies For Cost-Effective Smart Contracts*. (2025). Hacken. <https://hacken.io/discover/solidity-gas-optimization/>
4. Asim, U. (2025, November 19). *Solidity Gas Optimization: 12 Techniques to Make Your Smart Contracts Cheaper and More Efficient*. Alchemy.
<https://www.alchemy.com/overviews/solidity-gas-optimization>
5. *Optimize Gas on Base and Other L2 Chains With This Solidity Tutorial*. (2025). Cyfrin.io.
<https://www.cyfrin.io/blog/solidity-gas-efficiency-tips-tackle-rising-fees-base-other-12>